UNIVERSITY OF SCIENCE, VNU-HCM

Faculty of Information Technology

CSC14003 - Introduction to Artificial Intelligence

Project 2: Logic

Futoshiki Puzzles

Group Members

Nguyễn Khánh Toàn	MSSV: 24127252
Nguyễn Chí Tài	MSSV: 24127529
Nguyễn Ngọc Thiên	MSSV: 24127545
Phan Quang Tiến	MSSV: 24127250

Semester II (2025 - 2026)

Contents

1 Introduction **1**
- 1.1 Project Overview . . . 1
- 1.2 Objectives and Scope . . . 1
- 1.3 Team Information . . . 2

2 Project Planning and Task Distribution **2**
- 2.1 Work Breakdown . . . 2
- 2.2 Member Responsibilities . . . 2
- 2.3 Completion Percentage per Member . . . 2
- 2.4 Project Management Board . . . 2

3 Problem Formulation **2**
- 3.1 Puzzle Rules . . . 2
- 3.2 First-Order Logic Formalization . . . 3
  - 3.2.1 Predicate Symbols . . . 3
  - 3.2.2 Core Axioms . . . 3
  - 3.2.3 Step-by-step CNF Derivation . . . 4
- 3.3 Forward Chaining (DPLL-based SAT Solver) . . . 6
  - 3.3.1 Idea . . . 6
  - 3.3.2 Algorithm Analysis . . . 7
  - 3.3.3 Pseudocode . . . 10
  - 3.3.4 Illustration . . . 14
- 3.4 Backward Chaining (SLD Resolution) . . . 15
  - 3.4.1 Idea . . . 15
  - 3.4.2 Algorithm Analysis . . . 16
  - 3.4.3 Pseudocode . . . 18
  - 3.4.4 Illustration . . . 21
- 3.5 A* Search . . . 21
  - 3.5.1 Idea . . . 21
  - 3.5.2 Heuristic Function and Admissibility . . . 22
  - 3.5.3 Pseudocode . . . 23
  - 3.5.4 Illustration . . . 24
  - 3.5.5 Algorithm Analysis . . . 24

3.6 Brute-force and Backtracking 25
3.6.1 Idea 25
3.6.2 Pseudocode 25
3.6.3 Illustration 26
3.6.4 Algorithm Analysis 26
3.7 SAT Solver (Using SAT Library) 27
3.7.1 Idea 27
3.7.2 Constraint Encoding 27
3.7.3 Pseudocode 28
3.7.4 Illustration 28
3.7.5 Algorithm Analysis 29

4 Experiments and Evaluation 29
4.1 Experimental Setup 29
4.2 Input and Output Test Cases 30
4.3 Evaluation Metrics 30
4.3.1 Runtime 30
4.3.2 Memory Usage 30
4.3.3 Number of Inferences or Expansions 30
4.4 Comparative Analysis Across Algorithms 30
4.5 Self-evaluation Against Project Criteria 30

5 References 30

A Appendix 30
A.1 Pseudocode 30
A.2 Demo Video Links 31
A.3 Additional Tables and Charts 31
A.4 Submission Checklist 31

List of Tables

List of Figures

1. Introduction

1.1. Project Overview

This project implements a complete pipeline for solving *Futoshiki* logic puzzles in the context of CSC14003 — Introduction to Artificial Intelligence. A Futoshiki puzzle is an $N \times N$ grid where each row and column must contain numbers from 1 to N without repetition, while inequality symbols between adjacent cells must also be satisfied.

Our system is organized into clear modules: data I/O, domain modeling, knowledge-base construction, solver implementations, and a GUI for interaction and visualization. The codebase supports loading multiple input instances, running selected solving methods, and exporting results in a reproducible format. This report focuses on two representative approaches: A* search and SAT-based solving.

1.2. Objectives and Scope

The main objectives of this project are:

- Model Futoshiki constraints correctly and consistently for both search-based and logic-based solving.
- Implement and evaluate AI solving strategies with readable, modular, and reusable code.
- Build a practical interface for testing puzzle instances of different sizes and difficulties.
- Compare algorithm behavior through solution quality and computational efficiency.

Scope of this report:

- We present the overall project context.
- In the algorithm chapter, we provide detailed descriptions for only two methods: **A* Search** and **SAT Solver (using a SAT library)**.
- Other implemented methods are part of the codebase but are not expanded in detail here.

1.3. Team Information

The project was completed by four students from the Faculty of Information Technology, University of Science — VNU-HCM:

- Nguyễn Khánh Toàn — MSSV: 24127252
- Nguyễn Chí Tài — MSSV: 24127529
- Nguyễn Ngọc Thiên — MSSV: 24127545
- Phan Quang Tiến — MSSV: 24127250

The team collaborated on problem modeling, implementation, testing, and report writing, with task assignments summarized in the planning section.

2. Project Planning and Task Distribution

2.1. Work Breakdown

[Chèn nội dung tại đây]

2.2. Member Responsibilities

[Chèn nội dung tại đây]

2.3. Completion Percentage per Member

[Chèn nội dung tại đây]

2.4. Project Management Board

[Chèn nội dung tại đây]

3. Problem Formulation

3.1. Puzzle Rules

Futoshiki is a square-grid logic puzzle played on an $N \times N$ board. The primary objective is to fill the grid with digits such that each row and each column contains every number from 1 to N exactly once. The puzzle is governed by three fundamental categories of constraints:

1. **Latin Square Property:** Similar to Sudoku, every row and every column must be a permutation of the set $\{1, 2, \ldots, N\}$. This implies that no digit can be repeated within the same row or the same column.
2. **Inequality Constraints:** The defining characteristic of Futoshiki is the presence of "less than" ($<$) and "greater than" ($>$) signs between adjacent cells. These signs may be placed horizontally (between cells in the same row) or vertically (between cells in the same column). Any numerical assignment must strictly satisfy these relational constraints.
3. **Pre-filled Clues (Fixed Values):** At the start of the puzzle, some cells may already contain fixed digits (referred to as "Given Clues"). These values are immutable and serve as the starting points for the logical deduction process.

A puzzle is considered solved when every cell is filled with a valid digit and all structural, permutation, and inequality constraints are simultaneously satisfied. For an $N \times N$ board, the total number of cells to be determined is N^2, leading to a search space that grows factorially with N.

3.2. First-Order Logic Formalization

3.2.1 Predicate Symbols

The following vocabulary is used to formalize the Futoshiki puzzle rules into First-Order Logic sentences:

- $Val(i, j, v)$: Cell (i, j) is assigned value v
- $Given(i, j, v)$: Cell (i, j) has the pre-filled clue value v
- $LessH(i, j)$: Horizontal constraint indicating cell $(i, j) < (i, j + 1)$.
- $GreaterH(i, j)$: Horizontal constraint indicating cell $(i, j) > (i, j + 1)$.
- $LessV(i, j)$: Vertical constraint indicating cell $(i, j) < (i + 1, j)$.
- $GreaterV(i, j)$: Vertical constraint indicating cell $(i, j) > (i + 1, j)$.
- $Less(v_1, v_2)$: Integer $v1 < v2$ (background arithmetic relation)

3.2.2 Core Axioms

The logic of a Futoshiki puzzle of size NxN is encoded using the following universal axioms:

1. **Cell Completeness and Uniqueness:** Each cell must contain exactly one value.

- $\forall i \forall j \exists v\ Val(i, j, v)$
- $\forall i \forall j \forall v_1 \forall v_2 (Val(i, j, v_1) \wedge Val(i, j, v_2) \Rightarrow v_1 = v_2)$

2. **Value Domain and Boundary Constraints:** Values must be within the range $[1, N]$.
   - $\forall i \forall j \forall v (Val(i, j, v) \Rightarrow Less(0, v) \wedge Less(v, N + 1))$
3. **Permutation Constraints (Row and Column Uniqueness):** Every number appears exactly once in each row and column.
   - Row: $\forall i \forall j_1 \forall j_2 \forall v (Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2 \Rightarrow \perp)$
   - Column: $\forall i_1 \forall i_2 \forall j \forall v (Val(i_1, j, v) \wedge Val(i_2, j, v) \wedge i_1 \neq i_2 \Rightarrow \perp)$
4. **Inequality Constraints:** Satisfaction of all horizontal and vertical relational signs.
   - $\forall i \forall j \forall v_1 \forall v_2 (LessH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j{+}1, v_2) \Rightarrow Less(v_1, v_2))$
   - $\forall i \forall j \forall v_1 \forall v_2 (GreaterH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j{+}1, v_2) \Rightarrow Less(v_2, v_1))$
   - $\forall i \forall j \forall v_1 \forall v_2 (LessV(i, j) \wedge Val(i, j, v_1) \wedge Val(i{+}1, j, v_2) \Rightarrow Less(v_1, v_2))$
   - $\forall i \forall j \forall v_1 \forall v_2 (GreaterV(i, j) \wedge Val(i, j, v_1) \wedge Val(i{+}1, j, v_2) \Rightarrow Less(v_2, v_1))$
5. **Given Clues Enforcement:** Pre-filled digits must be assigned to their respective cells.
   - $\forall i \forall j \forall v (Given(i, j, v) \Rightarrow Val(i, j, v))$
6. **Background Arithmetic (Ordering Properties)** This axiom defines the strict ordering properties of the *Less* predicate:
   - **Irreflexivity:** $\forall v \neg Less(v, v)$
   - **Transitivity:** $\forall v_1 \forall v_2 \forall v_3 (Less(v_1, v_2) \wedge Less(v_2, v_3) \Rightarrow Less(v_1, v_3))$

   Note: For practical implementation, the Knowledge Base is populated with ground facts $Less(a, b)$ for all constant pairs (a, b) where a is numerically smaller than b within the domain $[1, N]$.

3.2.3 Step-by-step CNF Derivation

The following examples demonstrate the conversion of FOL axioms into Conjunctive Normal Form (CNF) for use in inference engines.

Example 1: Cell Value Uniqueness
FOL Form: $\forall i \forall j \forall v_1 \forall v_2 (Val(i, j, v_1) \wedge Val(i, j, v_2) \Rightarrow v_1 = v_2)$

1. **Step 1 (Eliminate Implication):** Apply the rule $A \Rightarrow B \equiv \neg A \vee B$:

$$\forall i \forall j \forall v_1 \forall v_2 (\neg (Val(i, j, v_1) \wedge Val(i, j, v_2)) \vee (v_1 = v_2))$$

2. **Step 2 (Move Negation Inward):** Apply De Morgan's Law to distribute the negation:

$$\forall i \forall j \forall v_1 \forall v_2 (\neg Val(i, j, v_1) \vee \neg Val(i, j, v_2) \vee (v_1 = v_2))$$

3. **Step 3 (Drop universal quantifiers):** Academic standard

$$\neg Val(i, j, v_1) \vee \neg Val(i, j, v_2) \vee (v_1 = v_2)$$

4. **Step 4 (Standardization for Solver) (Practical):** Since we only consider pairs where $v_1 \neq v_2$ during the grounding process, the literal $(v_1 = v_2)$ evaluates to False ($\perp$) and is removed. **CNF Clause:**

$$[\neg Val(i, j, v_1), \neg Val(i, j, v_2)] \quad \forall v_1 \neq v_2$$

Example 2: Row Uniqueness (Binary Clauses)
FOL Form: $\forall i \forall j_1 \forall j_2 \forall v (Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2 \Rightarrow \perp)$

1. **Step 1 (Eliminate Implication):**

$$\forall i \forall j_1 \forall j_2 \forall v (\neg (Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2) \vee \perp)$$

2. **Step 2 (Move Negation Inward):** Apply De Morgan's Law and remove the contradiction constant $\perp$:

$$\forall i \forall j_1 \forall j_2 \forall v (\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v) \vee (j_1 = j_2))$$

3. **Step 3 (Drop universal quantifiers):** Academic standard

$$\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v) \vee (j_1 = j_2)$$

4. **Step 4 (Refine Difference Constraint) (Practical):** Since the axiom iterates over distinct column indices, this literal is False and is discarded during grounding. **CNF Clause:**

$$[\neg Val(i, j_1, v), \neg Val(i, j_2, v)] \quad \forall j_1 \neq j_2$$

Example 3: Horizontal Less-than

FOL Form: $\forall i \forall j \forall v_1 \forall v_2 (LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j{+}1,v_2) \Rightarrow Less(v_1,v_2))$

1. **Step 1 (Eliminate Implication):**

$$\forall i \forall j \forall v_1 \forall v_2 (\neg(LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j{+}1,v_2)) \vee Less(v_1,v_2))$$

2. **Step 2 (Move Negation Inward):** Apply De Morgan's Law to distribute the negation across the conjunction:

$$\forall i \forall j \forall v_1 \forall v_2 (\neg LessH(i,j) \vee \neg Val(i,j,v_1) \vee \neg Val(i,j{+}1,v_2) \vee Less(v_1,v_2))$$

3. **Step 3 (Drop Universal Quantifiers):** Remove the quantifiers to obtain the final clausal form: **CNF Clause:**

$$[\neg LessH(i,j), \neg Val(i,j,v_1), \neg Val(i,j+1,v_2), Less(v_1,v_2)]$$

3.3. Forward Chaining (DPLL-based SAT Solver)

3.3.1 Idea

The forward-chaining solver developed by our team operates as a hybrid of propositional Unit Propagation (the grounded counterpart of FOL Forward Chaining) and DPLL backtracking search.

- **KB construction (*FOLKB*):** Converts a Futoshiki instance into a ground CNF Knowledge Base by fully instantiating all FOL axioms over the finite domain $\{1, \ldots, N\}$. Every cell-value pair (i, j, v) is mapped to a unique Boolean variable $\text{var}(i, j, v) = i \cdot N^2 + j \cdot N + v \in \{1, \ldots, N^3\}$.
- **Forward-chaining engine (*_propagate*):** Implements Unit Propagation over the ground Horn KB. Starting from the unit clauses that encode pre-filled cells and boundary inequalities, the engine repeatedly applies Modus Ponens: whenever all but one literal in a clause are falsified, the remaining literal is forced *True*. This is exactly FOL Forward Chaining restricted to the grounded domain.
- **Search controller (*_dpll*):** When Unit Propagation can no longer derive new facts, the controller selects an unassigned variable via the MRV + Frequency heuristic, branches on its two possible truth values, and backtracks upon conflict.

Why Unit Propagation equals Forward Chaining on a ground KB: The fundamental FOL axiom "every cell has at least one value" ($\forall i \, \forall j \, \exists v \; Val(i, j, v)$) contains an existential quantifier that cannot be expressed as a definite Horn rule in its general form. After *full grounding* over the finite domain $\{1, \ldots, N\}$, however, this axiom becomes the finite disjunction $\bigvee_{v=1}^{N} x_{i,j,v}$, which *is* a valid CNF clause. Consequently, every step of Unit Propagation on the ground KB is logically equivalent to one application of Modus Ponens on the corresponding grounded Horn rule — the defining operation of Forward Chaining.

Technical Implementation Notes:

- **0-based Indexing:** While the formal FOL axioms in this report use 1-based indexing (as per the mathematical specification $1..N$), the internal variable encoding uses 0-based row/column indices. This aligns naturally with Python's list structures and eliminates redundant ± 1 arithmetic at every node expansion.
- **Full Grounding vs. Partial Grounding:** Unlike the Backward Chaining solver which keeps some rules in first-order form, this solver grounds the entire KB upfront. For grid sizes $N \leq 9$ the resulting CNF (at most $N^3 \approx 729$ variables and $O(N^4)$ clauses) remains tractable and enables $O(1)$ variable lookup via a direct-indexed model array.
- **Literal-to-clause index:** An inverted index literal_to_clauses[ℓ] stores exactly those clauses that contain literal ℓ. When literal p is set *True*, only the clauses in literal_to_clauses[$-p$] need to be re-examined, reducing per-step cost from $O(|\mathcal{C}|)$ to $O(\deg(p))$.
- **Step Callback for visualisation:** step_callback(r, c, v) is invoked each time a cell is assigned ($v > 0$) or retracted ($v = 0$), enabling real-time GUI updates without coupling the solver to the display layer.

3.3.2 Algorithm Analysis

CNF Encoding — Variable Schema

Each Boolean variable encodes one cell-value pair:

$$\text{var}(i,\, j,\, v) \;=\; i \cdot N^2 + j \cdot N + v, \qquad i, j \in \{0, \ldots, N-1\}, \quad v \in \{1, \ldots, N\}.$$

The model array $model[1..N^3]$ stores $\{-1, 0, +1\}$ for *False*, *unassigned*, and *True* respectively.

CNF Clause Groups

The ground KB is built from nine constraint groups:

Group	Clause form	Constraint
A1	$(x_{i,j,1} \vee \cdots \vee x_{i,j,N})$	At-least-one per cell
A2	$(\neg x_{i,j,a} \vee \neg x_{i,j,b}),\ a < b$	At-most-one per cell
A3	$(\neg x_{i,j_1,v} \vee \neg x_{i,j_2,v}),\ j_1 < j_2$	Row uniqueness
A4	$(\neg x_{i_1,j,v} \vee \neg x_{i_2,j,v}),\ i_1 < i_2$	Column uniqueness
A5/A6	$(\neg x_{i,j,a} \vee \neg x_{i,j+1,b})$ if $a \not< b$	Horizontal $<$ / $>$
A7/A8	$(\neg x_{i,j,a} \vee \neg x_{i+1,j,b})$ if $a \not< b$	Vertical $\wedge$ / $\vee$
A9	$(x_{i,j,v})$	Pre-filled clues
A10	*(implicit in encoding)*	Domain $[1..N]$
A11	*(implicit, via negation)*	Inequality via $\neg(a < b)$

Unit Propagation (Forward Chaining step)

When literal p is assigned *True*, every clause in literal_to_clauses$[-p]$ is inspected:

- If the clause already contains a *True* literal: skip.
- If exactly one literal ℓ remains unassigned: add ℓ to the agenda (Modus Ponens / unit propagation step).
- If no unassigned literal remains and the clause is unsatisfied: **conflict** — return *False*.

Variable Selection Heuristic: MRV + Frequency

When Unit Propagation halts without completing the assignment, the solver must branch. Variable selection proceeds in two levels:

1. **Minimum Remaining Values (MRV):** Among all unassigned cells, choose the cell (i, j) whose *domain size* $|\{v : model[\mathrm{var}(i, j, v)] \neq -1\}|$ is smallest. This *fail-first* principle detects dead ends as early as possible.
2. **Frequency tiebreaker:** Among all domain literals of the chosen cell, select the literal with the highest frequency in the CNF — assigning a frequent literal satisfies more clauses simultaneously.

Complexity Analysis

Aspect	Bound	Remark
Variables	N^3	One Boolean per cell-value pair
Clauses (A1)	N^2	One per cell
Clauses (A2)	$N^2\binom{N}{2}$	Pairwise exclusion per cell
Clauses (A3+A4)	$2N\binom{N}{2}N$	Row and column uniqueness
Max recursion depth	$\leq N^2$	One cell resolved per level
Time (worst case)	$O(N^3 \cdot 2^{N^3})$	Full Boolean search space
Time (average)	$O(N^3 \cdot C)$	C = actual backtracks (small)
Space	$O(N^3)$	Model array + agenda

Advantages and Limitations

- **Complete:** Backtracking guarantees a solution is found if one exists; the solver never misses a valid assignment.
- **Efficient propagation:** The inverted index restricts each propagation step to $O(\deg(p))$ clause checks instead of $O(|\mathcal{C}|)$.
- **Low memory footprint:** Only the flat model array and a small agenda deque are maintained; no explicit clause deletion is performed.
- **No clause learning (CDCL):** The solver does not learn *conflict clauses*, unlike modern industrial SAT solvers (MiniSat, Glucose). Repeated conflicts in similar sub-trees are not avoided.
- **Static index:** literal_to_clauses is built once and not updated; satisfied clauses are still visited during propagation, incurring a minor constant overhead.

3.3.3 Pseudocode

Algorithm 1 Master Coordinator

```
function SOLVE(puzzle)
    kb ← FOLKB(puzzle)
    clauses ← kb.BUILD_KB
    max_var ← N^3
    lit_to_clauses ← BUILDINDEX(clauses)
    freq ← COUNTFREQUENCY(clauses)
    model ← array[0 ... max_var] initialised to 0
    agenda ← DEQUE
    for each clause c ∈ clauses with |c| = 1 do
        agenda.APPEND(c[0])
    end for
    result ← DPLL(clauses, lit_to_clauses, agenda, model)
    if result ≠ None then
        return TRANSLATETOGRID(result)
    else
        return None                                  ▷ Puzzle has no solution
    end if
end function
```

Algorithm 2 DPLL — Recursive Branch-and-Propagate

```
function DPLL(clauses, index, agenda, model)
    nodes_expanded += 1
    if not PROPAGATE(clauses, index, agenda, model) then
        return None                                  ▷ Conflict — backtrack
    end if
    best_var ← GETUNASSIGNEDVAR(model)
    if best_var = None then
        return model                ▷ All variables assigned — solution found
    end if
    agenda_true ← DEQUE([best_var])
    result ← DPLL(clauses, index, agenda_true, model)
    if result ≠ None then
        return result
    end if
                                        ▷ Branch False — restore snapshot
    model ← saved
    agenda_false ← DEQUE([−best_var])
    return DPLL(clauses, index, agenda_false, model)
end function
```

Algorithm 3 Unit Propagation — Forward Chaining Engine

```
function PROPAGATE(clauses, index, agenda, model)
    while agenda is not empty do
        p ← agenda.POPLEFT
        if ISFALSE(p, model) then
            return False                                  ▷ Immediate conflict
        end if
        if not ISASSIGNED(|p|, model) then
            ASSIGN(p, model)                ▷ Modus Ponens: set literal True
            if callback ≠ None and p > 0 then
                CALLBACK(Decode(p))                       ▷ Notify GUI
            end if
        end if
        for each clause c ∈ index[−p] do
            sat ← False;  unassigned ← 0;  unit ← 0
            for each literal ℓ ∈ c do
                if ISTRUE(ℓ, model) then
                    sat ← True; break
                end if
                if not ISASSIGNED(|ℓ|, model) then
                    unassigned += 1;  unit ← ℓ
                    if unassigned > 1 then break
                    end if
                end if
            end for
            if sat then continue
            end if
            if unassigned = 0 then return False           ▷ CONFLICT
            end if
            if unassigned = 1 and unit ∉ agenda_set then
                agenda.APPEND(unit)                       ▷ New unit clause
            end if
        end for
    end while
    return True
end function
```

Algorithm 4 Variable Selection — MRV + Frequency Heuristic

```
function GetUnassignedVar(model)
    best_lit ← None;  best_dom ← ∞;  best_freq ← −1
    for i ∈ 0 ... N − 1 do
        for j ∈ 0 ... N − 1 do
            if ∃ v : model[var(i, j, v)] = 1 then continue    ▷ Cell already assigned
            end if
            dom ← [var(i, j, v) | v ∈ 1..N,  model[var(i, j, v)] ≠ −1]
            if dom = ∅ then continue
            end if
            cand ← arg max_{ℓ∈dom}  freq[ℓ]
            f ← freq[cand]
            if |dom| < best_dom or (|dom| = best_dom and f > best_freq) then
                best_dom ← |dom|;  best_lit ← cand;  best_freq ← f
            end if
        end for
    end for
    return best_lit                    ▷ None if all cells are assigned
end function
```

Algorithm 5 Index Construction and Frequency Count

```
function BuildIndex(clauses)
    index ← defaultdict(list)
    for each clause c ∈ clauses do
        for each literal ℓ ∈ c do
            index[ℓ].append(c)
        end for
    end for
    return index
end function
function CountFrequency(clauses)
    freq ← defaultdict(0)
    for each clause c ∈ clauses do
        for each literal ℓ ∈ c do
            freq[|ℓ|] += 1
        end for
    end for
    return freq
end function
function TranslateToGrid(model)
    grid ← N × N matrix of 0
    for i ∈ 0..N − 1 do
        for j ∈ 0..N − 1 do
            for v ∈ 1..N do
                if model[var(i, j, v)] = 1 then
                    grid[i][j] ← v; break
                end if
            end for
        end for
    end for
    return grid
end function
```

3.3.4 Illustration

[Chèn nội dung tại đây]

3.4. Backward Chaining (SLD Resolution)

3.4.1 Idea

The backward-chaining solver developed by our team operates as a hybrid of Prolog-style SLD resolution and backtracking search.

- **KB construction($HornConverter$):** Converts a Futoshiki instance into a Knowledge Base (KB) consisting of facts and definite Horn rules over predicates like $val(i, j, v)$ and $not_val(i, j, v)$.
- **Backward chaining engine (fol_bc_or / fol_bc_and):** Prolog-like interpreter implemented from scratch that answers queries using unification and depth-first goal reduction.
- **Search controller ($_backtrack$):** Responsible for filling the grid. For each empty cell (r, c), the controller calls the backward chaining engine to enumerate valid candidates for $val(r, c, v)$. It then asserts the chosen val as a new fact in the KB, recurses to the next cell, and retracts the fact upon failure.

Why search is integrated with SLD Resolution: The fundamental First-Order Logic (FOL) constraint for Futoshiki states that "every cell has at least one value" ($\forall i \forall j \exists v\ Val(i, j, v)$). This existential quantification is not expressible as a definite Horn rule. Therefore, a pure Prolog-style engine cannot natively generate assignments without complex, inefficient workarounds. To resolve this, we employ choice + backtracking in Python to build a full assignment incrementally, while utilizing the Horn rules within the BC engine purely to derive exclusions (not_val) and prune invalid branches.

Technical Implementation Notes:

- **0-based Indexing:** While the formal FOL axioms in this report use 1-based indexing (as per the mathematical specification $1..N$), the internal engine and the trace examples below utilize 0-based indexing. This engineering decision aligns naturally with Python's List data structures, eliminating redundant $+1/-1$ arithmetic at every node expansion and optimizing low-level computational speed.
- **Hybrid vs. Pure Prolog:** A pure Prolog implementation solving a 7×7 or higher grid via a single query would generate an SLD tree so

deep it would cause a RecursionError or a memory blowout in Python. The Hybrid architecture ensures the memory footprint remains manageable by using BC as a highly precise "Domain Pruning Oracle" for individual cells, rather than a global solver.

- **Partial Grounding:** To mitigate combinatorial explosion during unification, our implementation uses Partial Grounding. Static constraints (like boundary inequalities) are fully grounded into facts (e.g., $not_val(0, 1, 3)$), while the core generator rules are kept in First-Order form with variables (e.g., $val(i, j, v)$). This drastically reduces the size of the KB while retaining the dynamic power of SLD resolution.

3.4.2 Algorithm Analysis

Atoms (Predicates)

The Horn KB used the following vocabulary:

- $val(i, j, v)$: Cell (i, j) is assigned the value v.
- $given(i, j, v)$: Cell (i, j) has the pre-filled clue value v
- $less_h(i, j)$ / $greater_h(i, j)$: Horizontal inequalities between (i, j) and $(i, j + 1)$.
- $less_v(i, j)$ / $greater_v(i, j)$: Vertical inequalities between (i, j) and $(i + 1, j)$.
- $less(v1, v2)$: Background arithmetic fact representing $v_1 < v_2$.
- $neq(x, y)$: Background inequality fact representing $x \neq y$.
- $not_val(i, j, v)$: Value v is explicitly forbidden at cell (i, j).

Facts

- **Background facts**: All valid $less(v1, v2)$ and $neq(x, y)$ combinations within the finite domain $1..N$.
- **Instance facts**: $given(...)$ clues, inequality signs, and immediate "boundary pruning" facts ($not_val(...)$) derived directly from extreme inequality conditions (e.g., a cell on the smaller side of an inequality cannot hold the maximum value N).

Rules (Definite Horn)

Constraints are strictly encoded as rules deriving not_val:

- Given $\Rightarrow Val : val(i, j, v) : -given(i, j, v)$

- Cell uniqueness: If a value is chosen, all others are forbidden in that cell.
- Row/Col uniqueness: If $val(i, j1, v)$ holds, then $not_val(i, j2, v)$ for all $j_2 \neq j_1$.
- Inequalities: Values that violate a $<$ or $>$ constraint trigger a not_val derivation.

Existence / Choice Rule via NAF

To generate valid candidates for an empty cell, the following rule is injected into the KB:$val(i, j, v) : -domain(v), naf(not_val(i, j, v))$ Explain: A value v is a valid candidate for cell (i, j) IF it exists in the domain AND the system fails to prove that it is forbidden (not_val). This utilizes "Negation as Failure" (NAF), enabling non-monotonic reasoning over our finite Knowledge Base.

Unification and Substitution

- $unify(x, y, \theta)$: Computes the Most General Unifier (MGU) for two expressions, returning the substitution dictionary θ or "failure".
- $subst(\theta, expr)$: Applies the substitution θ to goals and rule bodies. Variables are represented as lowercase strings (e.g., 'v', 'i', 'j').

SLD Resolution (AND/OR Search Tree)

- OR node (bc_or): Proves a goal by attempting to match and unify it against the head of any available fact or rule.
- AND node (bc_and): Proves the body of a rule by proving all subgoals sequentially from left to right.
- This mimics the standard Prolog evaluation strategy: depth-first, left-most.

Negation as Failure (NAF) Handling

- naf(G) succeeds if and only if the engine exhausts all paths and cannot derive G.
- Safety Implementation: When evaluating inside a NAF block ($in_naf = True$), the engine restricts derivations for val to facts only, ignoring rules. This prevents fatal self-supporting cycles (e.g., attempting to

prove val by assuming not not_val, which loops back to val), ensuring termination and soundness.

Optimizations Implemented

- Fact and Rule Indexing: Knowledge is partitioned using dictionaries ($fact_index[pred], pred_rule_index, exact_rule_index$) to ensure $O(1)$ lookup times instead of scanning the entire KB sequentially.Per-query
- Memoization: Subgoal results are cached (self.memo) during a single bc_ask lifecycle to prevent recomputation of identical states.
- Cycle Detection (visited set): Tracks the active resolution path to prevent infinite recursion in cyclic goal graphs.
- Boundary Pruning: Converting extreme inequalities into immediate not_val facts drastically reduces the search space before resolution even begins.

3.4.3 Pseudocode

Algorithm 6 Master Coordinator

```
function SOLVE(puzzle)
    kb ← HornConverter(puzzle).build()
    add facts domain(1..N)
    add facts val(...)
    for givens in the grid do
        add rule schema: val(i, j, v) : −domain(v), naf(not_val(i, j, v))
        build indexes for facts/rules
    end for

    for each given (r, c) = v do
        if FOL_BC_ASK(kb, not_val(r, c, v) succeeds then
            return failure // Contradiction in givens
        end if
        return _BACKTRACK(kb)
    end for
end function
```

Algorithm 7 State Space Manager

```
function BACKTRACK(kb)
    if grid complete then
        return success
    end if
    (r, c) ← first empty cell
    for each answer substitution theta in FOL_BC_ASK(kb, val(r, c, v)) do
        value ← theta[v]
        grid[r][c] ← value
        assert fact val(r, c, value) into kb
        if _BACKTRACK(kb) success then
            return success
        end if
        retract fact val(r, c, value) from kb
        grid[r][c] ← 0
    end for
    return failure
end function
```

Algorithm 8 Inference Interface

```
function FOL_BC_ASK(kb, goal)
    memo ← ∅                                        ▷ Init Memoization
    return FOL_BC_OR(kb, goal, theta = ∅, in_naf = false, visited = ∅)
end function
```

Algorithm 9 Goal Resolver - OR Node

```
function FOL_BC_OR(kb, goal, theta, in_naf, visited)
    g ← SUBST(θ, goal)
    if (g, in_naf) ∈ visited then
        return ∅                                        ▷ Cycle detection
    end if
    visited′ ← visited ∪ {(g, in_naf)}
    if g is naf(inner) then                             ▷ Negation As Failure
        if FOL_BC_OR(kb, inner, theta, true, visited′) is None then
            return {theta}                              ▷ NAF success
        else
            return ∅                                    ▷ NAF failure
        end if
    end if
    answers ← ∅
    for each fact F matching g in kb do
        theta′ ← UNIFY(args(g), args(head(F)), theta)
        if theta′ ≠ failure then
            answers ← answers∪FOL_BC_AND(kb, body(F), theta′, in_naf, visited′)
        end if
    end for
    if not (in_naf and pred(g) == val) then
        for each rule R matching g in kb do
            theta′ ← UNIFY(args(g), args(head(R)), theta)
            if theta′ ≠ failure then
                answers ← answers∪FOL_BC_AND(kb, body(R), theta′, in_naf, visited′)
            end if
        end for
    end if
    return answers
end function
```

Algorithm 10 Rule Body Validator - AND Node

```
function FOL_BC_AND(kb, goals, theta, in_naf, visited)
    if goals is None then
        return {theta}                ▷ Chứng minh thành công toàn bộ Body
    end if
    first ← SUBST(theta, goals[0])
    rest ← goals[1 :]                 ▷ Các mục tiêu còn lại
    answers ← ∅
    for each theta′ in FOL_BC_OR(kb, first, theta, in_naf, visited) do
        answers ← answers∪FOL_BC_AND(kb, rest, theta′, in_naf, visited)
    end for
    return answers
end function
```

3.4.4 Illustration

[Chèn nội dung tại đây]

3.5. A* Search

3.5.1 Idea

In this project, each search state is a partial Futoshiki assignment, where value 0 denotes an unfilled cell. The A* solver expands states according to

$$f(s) = g(s) + h(s),$$

with the following cost model:

- Each action assigns exactly one value to one empty cell, so each step has unit cost.
- $g(s)$ is the number of assignments already made from the initial state.
- $h(s)$ estimates the minimum number of assignments still needed to reach a complete solution.

The transition model assigns one value to one unfilled cell. To reduce branching, the solver selects the next variable by MRV (Minimum Remaining Values), i.e., the cell with the smallest candidate domain. Before evaluating child states, domains are filtered by row/column uniqueness and inequality constraints; optional AC-3 propagation is then applied to remove unsupported values.

This design combines informed search (A*) with CSP-style filtering (MRV + AC-3), making the exploration significantly more directed than naive uninformed expansion.

3.5.2 Heuristic Function and Admissibility

Let $U(s)$ be the number of unassigned cells in state s. We choose:

$$h(s) = U(s).$$

Equivalently, if N is board size and $A(s)$ is number of assigned cells, then

$$h(s) = N^2 - A(s).$$

Feasibility pruning with AC-3 is used as a separate consistency check: if propagation yields an empty domain, the state is discarded as inconsistent.

Why this heuristic is admissible:

- From a partial state with $U(s)$ empty cells, any complete solution must assign all those $U(s)$ cells.
- Each action can assign at most one cell.
- Therefore the true remaining cost $h^*(s)$ satisfies $h^*(s) \geq U(s) = h(s)$.

Hence $h(s)$ never overestimates the optimal remaining cost, so it is admissible.

Consistency (monotonicity):

- After one valid assignment $s \to s'$, we have $U(s') = U(s) - 1$.
- With unit step cost $c(s, s') = 1$:

$$h(s) = U(s) = 1 + U(s') = c(s, s') + h(s').$$

- So h is consistent as well.

3.5.3 Pseudocode

Algorithm 11 A* for Futoshiki (with MRV and AC-3)

```
Build initial state s_0 from puzzle
Validate givens; if invalid then return failure
Compute domains and heuristic for s_0
Initialize closed as an empty set
Initialize best_g[s_0]
Push s_0 into priority queue by key (f, h, −g)
while priority queue not empty do
    Pop best state s
    if s ∈ closed then
        continue
    end if
    Add s to closed
    if s is goal then return solved grid
    end if
    Select unassigned cell (r, c) by MRV
    for each value v in domain(r, c) do
        if assigning v is consistent then
            Create child state s′
            if s′ ∈ closed then
                continue
            end if
            Compute g(s′)
            if best_g[s′] exists and g(s′) is not better then
                continue
            end if
            Recompute domains (and AC-3 if enabled)
            if no empty domain in s′ then
                Set g(s′) = g(s) + 1, evaluate h(s′), f(s′) = g(s′) + h(s′)
                Update best_g[s′]
                Push s′ into priority queue
            end if
        end if
    end for
end while
return failure
```

3.5.4 Illustration

Consider a 4×4 instance. After loading givens and inequalities, the solver:

1. Initializes domains for all empty cells.
2. Runs consistency filtering; if AC-3 is enabled, unsupported values are removed transitively.
3. Chooses the most constrained empty cell by MRV.
4. Expands candidate assignments in priority order according to $f = g+h$.

Example progression:

- State s_0: 4 cells fixed from clues, several constrained inequalities.
- State s_1: assign value to the MRV cell (domain size 1 or 2), causing further domain reduction.
- State s_2: another forced assignment appears after propagation.
- Goal state: all cells filled, row/column uniqueness and inequalities all satisfied.

This illustrates how propagation shrinks the search tree before deep branching happens.

3.5.5 Algorithm Analysis

Let N be puzzle size (grid N^2 cells), and let b be average branching factor after filtering.

Time complexity (worst case):

- Search may still be exponential in the number of unassigned cells, roughly $O(b^d)$ with depth $d \leq N^2$.
- AC-3 propagation on each evaluated state adds polynomial overhead; practical runtime is dominated by number of expanded states.

Space complexity:

- A* stores open/closed states and domain data, so memory usage is higher than depth-first methods.
- In worst case, space is also exponential in search depth.

Practical behavior:

- The heuristic $h(s) = U(s)$ is admissible and easy to justify, which is suitable for the project requirement on heuristic correctness.
- Because this heuristic is simple, most practical speedup comes from MRV ordering and AC-3 pruning.

- The *closed* set avoids re-expanding visited states, and *best_g* avoids keeping inferior paths to the same state.
- The method is robust for medium-sized puzzles, but memory pressure can increase for larger or weakly constrained instances.

3.6. Brute-force and Backtracking

3.6.1 Idea

Brute-force is a straightforward approach that tries all possible assignments for the empty cells in the Futoshiki puzzle. For a puzzle of size $N \times N$, each empty cell can take a value from 1 to N. The algorithm generates all possible combinations and checks whether they satisfy the constraints (row, column, and inequality constraints). However, this approach is highly inefficient due to the exponential number of possibilities.

Backtracking is an optimized version of brute-force. Instead of generating all possible assignments, it builds the solution step by step and abandons a partial assignment as soon as it violates any constraint. At each step, the algorithm selects an empty cell, tries a valid value, and recursively proceeds to the next cell. If a conflict occurs, it backtracks and tries another value. This significantly reduces the search space compared to brute-force.

3.6.2 Pseudocode

Brute-force:

Algorithm 12 Brute-force

```
function BRUTE_FORCE(grid):
    empty_cells = list of all empty positions
    for each possible assignment of values to empty_cells:
        fill grid with assignment
        if is_valid(grid):
            return grid
    return failure
```

Backtracking:

Algorithm 13 Backtracking

```
function BACKTRACK(grid):
    if no empty cell:
        return grid

    (row, col) = find an empty cell

    for value from 1 to N:
        if is_valid(row, col, value):
            grid[row][col] = value

            result = BACKTRACK(grid)
            if result != failure:
                return result

            grid[row][col] = empty

    return failure
```

3.6.3 Illustration

Consider a 4×4 Futoshiki puzzle with some empty cells.

Brute-force will generate all possible combinations for the empty cells. For example, if there are k empty cells, it will try up to N^k assignments, checking each one against all constraints.

Backtracking, on the other hand, fills the grid step by step. Suppose it assigns a value that violates a constraint (e.g., duplicates in a row or breaks an inequality). Instead of continuing, it immediately backtracks and tries another value. This pruning avoids exploring invalid branches of the search tree, making it much more efficient in practice.

3.6.4 Algorithm Analysis

Brute-force Complexity: The time complexity is $O(N^k)$, where k is the number of empty cells. In the worst case (empty grid), this becomes $O(N^{N^2})$, which is infeasible for large N.

Backtracking Complexity: The worst-case time complexity is still exponential, but significantly reduced in practice due to pruning. The actual

performance depends on how early invalid assignments are detected.

Space Complexity: Backtracking uses recursion, so the space complexity is $O(N^2)$ for the grid plus $O(N^2)$ for the recursion stack in the worst case.

Comparison:

- Brute-force explores all possibilities without pruning.
- Backtracking prunes invalid states early, making it much more efficient.
- Backtracking is the preferred method for solving constraint satisfaction problems like Futoshiki.

3.7. SAT Solver (Using SAT Library)

3.7.1 Idea

The SAT approach transforms the Futoshiki puzzle into a propositional satisfiability problem in CNF (Conjunctive Normal Form). Instead of searching directly over puzzle states, we encode all puzzle rules as logical clauses and let a SAT engine decide whether there exists a satisfying assignment.

In this project, CNF clauses are generated by the `FOLKB` module, then solved by PySAT (Glucose3). If SAT, the returned model is decoded back to an $N \times N$ grid; if UNSAT, the puzzle has no valid solution under given constraints.

3.7.2 Constraint Encoding

For each triple (i, j, v), define boolean variable $X_{i,j,v}$:

$$X_{i,j,v} = \text{true} \iff \text{cell } (i, j) \text{ has value } v.$$

Variable ID mapping used in code:

$$\text{ID}(i, j, v) = i \cdot N^2 + j \cdot N + v,$$

with $i, j \in [0, N-1]$, $v \in [1, N]$, so IDs are in $[1, N^3]$.

Main CNF groups:

- **Cell at least one value:** each cell must take some $v \in \{1, \ldots, N\}$.
- **Cell at most one value:** no cell can take two different values simultaneously.

- **Row uniqueness:** two different columns in same row cannot share the same value.
- **Column uniqueness:** two different rows in same column cannot share the same value.
- **Given clues:** pre-filled cells are encoded as unit clauses.
- **Inequality constraints:** for each adjacent pair with sign $<$ or $>$, all violating value-pairs are forbidden by binary clauses.

This encoding guarantees that any satisfying model corresponds to a valid Futoshiki solution.

3.7.3 Pseudocode

Algorithm 14 SAT-based Futoshiki Solver

```
Build CNF clauses from puzzle constraints using FOLKB
Create SAT solver instance (Glucose3)
Add all CNF clauses to solver
if solver returns UNSAT then
    return failure
end if
Get SAT model (set of literals)
Initialize empty grid N × N
for each positive literal ℓ in model do
    if 1 ≤ ℓ ≤ N^3 then
        Decode ℓ → (i, j, v)
        Set grid(i, j) ← v
    end if
end for
if any cell remains unassigned then
    return failure
else
    return solved grid
end if
```

3.7.4 Illustration

For a 4×4 puzzle:

1. The encoder creates variables $X_{i,j,v}$ for all $4^3 = 64$ combinations.

2. Clauses are generated for Latin constraints (row/column uniqueness), given clues, and inequality symbols.
3. Glucose3 finds a satisfying model, for example including literals that decode to assignments such as $(0, 0) = 2$, $(0, 1) = 4$, etc.
4. After decoding all positive in-range literals, the grid is fully reconstructed and validated as complete.

Compared with incremental state expansion, the SAT method delegates global consistency reasoning to a highly optimized conflict-driven SAT engine.

3.7.5 Algorithm Analysis

Encoding cost:

- Number of variables is N^3.
- Number of clauses is polynomial in N, dominated by pairwise exclusivity and inequality forbidding constraints.

Solving cost:

- SAT is NP-complete in general, so worst-case complexity is exponential.
- In practice, modern CDCL solvers (such as Glucose3) are often very effective on structured constraints like Futoshiki.

Space and implementation trade-offs:

- Requires storing full CNF and SAT internal structures.
- Provides a clean and extensible formulation: adding new logical constraints usually means adding new clause templates.

Practical behavior in this project:

- SAT solver gives deterministic feasibility checking (SAT/UNSAT).
- Performance is strong on many instances, especially when constraints are dense enough to guide conflict learning.

4. Experiments and Evaluation

4.1. Experimental Setup

[Chèn nội dung tại đây]

4.2. Input and Output Test Cases

[Chèn nội dung tại đây]

4.3. Evaluation Metrics

[Chèn nội dung tại đây]

4.3.1 Runtime

[Chèn nội dung tại đây]

4.3.2 Memory Usage

[Chèn nội dung tại đây]

4.3.3 Number of Inferences or Expansions

[Chèn nội dung tại đây]

4.4. Comparative Analysis Across Algorithms

[Chèn nội dung tại đây]

4.5. Self-evaluation Against Project Criteria

[Chèn nội dung tại đây]

5. References

[Chèn nội dung tại đây]

A. Appendix

A.1. Pseudocode

[Chèn nội dung tại đây]

A.2. Demo Video Links

[Chèn nội dung tại đây]

A.3. Additional Tables and Charts

[Chèn nội dung tại đây]

A.4. Submission Checklist

[Chèn nội dung tại đây]